



INFORME DE LA PRACTICA DE LABORATORIO

I. PORTADA

Tema:	SW-AyLP-Trabajo-2.7: Sistema de Gestión Arreglos.
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Primero Software “B”
Alumno:	Sánchez Lema Isaac Adrián
	
Asignatura:	ALGORITMOS Y LÓGICA DE PROGRAMACIÓN
Docente:	Ing. José Caiza, Mg.

SISTEMA DE GESTIÓN Y ANÁLISIS DE CALIFICACIONES

1. INTRODUCCIÓN

En el contexto de las instituciones educativas, el procesamiento de calificaciones constituye una tarea crítica que, cuando se ejecuta manualmente, está sujeta a errores de cálculo, demoras y dificultades para la consulta individual de datos. La automatización de este proceso mediante herramientas de software permite no solo reducir el margen de error, sino también agilizar la toma de decisiones pedagógicas basadas en datos estadísticos precisos y disponibles en tiempo real.

La presente práctica de laboratorio propone el diseño e implementación de un **Sistema de Gestión y Análisis de Calificaciones** en lenguaje C++, orientado a procesar las notas del examen final de un grupo de estudiantes. El sistema abarca tres funcionalidades esenciales: el registro de calificaciones, la generación de un reporte estadístico descriptivo y la búsqueda secuencial de estudiantes por nombre.

Desde una perspectiva metodológica, el ejercicio integra de forma progresiva todas las etapas del ciclo de desarrollo de software aplicado a la programación estructurada: análisis del problema, diseño del algoritmo, pseudocódigo, diagrama de flujo, codificación en C++ y verificación mediante pruebas de escritorio. Este enfoque garantiza que el estudiante no solo produzca código funcional, sino que comprenda a profundidad la lógica subyacente a cada decisión de diseño.

La relevancia de esta práctica trasciende el ámbito académico inmediato, ya que los conceptos aplicados —arreglos, funciones, estructuras de control, algoritmos de búsqueda y estadística descriptiva— constituyen la base sobre la cual se construyen sistemas de información más complejos en el ejercicio profesional de la ingeniería de software.

2. OBJETIVOS

2.1 Objetivo General

Desarrollar un programa en lenguaje C++ que implemente un Sistema de Gestión y Análisis de Calificaciones, aplicando la metodología completa de desarrollo de software estructurado —análisis, algoritmo, pseudocódigo, diagrama de flujo, codificación y pruebas de escritorio— para resolver de manera eficiente el procesamiento estadístico y la consulta de datos académicos de un grupo de estudiantes.

2.2 Objetivos Específicos

- **Analizar** los requerimientos del sistema identificando con precisión las entradas, procesos y salidas necesarias, y definiendo las variables y estructuras de datos apropiadas en el entorno C++.



- **Diseñar** el algoritmo lógico y el pseudocódigo del sistema, estructurando las tres funcionalidades principales —registro, reporte estadístico y búsqueda— en módulos independientes y coherentes.
- **Implementar** el programa en C++ empleando arreglos paralelos, estructuras de control (condicionales y ciclos), funciones modularizadas y mecanismos de entrada/salida por consola.
- **Aplicar** el algoritmo de búsqueda secuencial para localizar registros de estudiantes por nombre, gestionando correctamente los casos de éxito y de registro no encontrado.
- **Verificar** la corrección del programa mediante pruebas de escritorio que validen los resultados estadísticos obtenidos y el comportamiento esperado de la búsqueda.

3. MARCO TEÓRICO

3.1 Programación Estructurada en C++

La programación estructurada es un paradigma que organiza el flujo de ejecución de un programa exclusivamente mediante tres estructuras de control fundamentales: secuencia, selección e iteración [1]. C++ es un lenguaje de programación compilado, tipado estáticamente y de propósito general que soporta este paradigma de forma nativa a través de sus características procedurales, al tiempo que extiende sus capacidades hacia la programación orientada a objetos y la programación genérica [2].

Para la presente práctica se utilizan exclusivamente las características procedurales del lenguaje: tipos de datos primitivos (int, float, string, bool), arreglos estáticos, funciones con paso de parámetros y las estructuras de control if-else, for, while y do-while.

3.2 Arreglos y Estructuras de Datos

Un arreglo (*array*) es una estructura de datos homogénea que almacena una colección de elementos del mismo tipo en posiciones de memoria contiguas, indexadas desde cero [3]. La declaración tipo nombre[tamaño] reserva en memoria estática un bloque continuo de tamaño elementos del tipo especificado.

En este sistema se emplean **arreglos paralelos**: dos arreglos de igual dimensión donde el índice *i* referencia simultáneamente al mismo estudiante en ambas estructuras (nombres[*i*] y notas[*i*]). Este patrón permite asociar información heterogénea sin recurrir a estructuras (struct) u objetos, siendo apropiado para el nivel introductorio de la práctica.

3.3 Modularización y Funciones

La modularización es el principio de diseño que consiste en descomponer un programa en unidades funcionales independientes, denominadas funciones o subprogramas, cada una con una responsabilidad única y bien definida [1]. Este principio mejora la legibilidad, facilita el mantenimiento, promueve la reutilización de código y reduce la complejidad cognitiva del desarrollo.

En C++, una función se define con la siguiente sintaxis:

```
tipo_retorno nombreFuncion(tipo_param1 param1, ...) {  
    // cuerpo de la función  
    return valor; // opcional según tipo_retorno  
}
```

Para este sistema se definen tres funciones principales: registrarCalificaciones(), mostrarReporte() y buscarEstudiante(), organizadas bajo un menú controlado por main().



3.4 Algoritmo de Búsqueda Secuencial

La búsqueda secuencial (o lineal) es el algoritmo más elemental para localizar un elemento dentro de una colección [4]. Consiste en recorrer el arreglo elemento por elemento desde el índice 0 hasta $n-1$, comparando cada entrada con el criterio de búsqueda. El algoritmo finaliza al encontrar la primera coincidencia (*best case* $O(1)$) o al agotar todos los elementos sin hallarla (*worst case* $O(n)$).

```
PARA i DESDE 0 HASTA n-1
  SI nombres[i] == nombreBuscado
    RETORNAR i // índice encontrado
  FIN SI
FIN PARA
RETORNAR -1 // no encontrado
```

Aunque su complejidad $O(n)$ no lo hace óptimo para grandes volúmenes de datos, resulta completamente adecuado para grupos académicos de tamaño reducido y no exige que los datos estén previamente ordenados [4].

3.5 Estadística Descriptiva Aplicada

El sistema calcula tres métricas de estadística descriptiva elemental sobre el conjunto de calificaciones [5]:

Métrica	Definición	Fórmula
Promedio aritmético	Medida de tendencia central	$\bar{x} = \Sigma \text{notas}[i] / n$
Conteo de aprobados	Frecuencia absoluta con nota ≥ 7	$\Sigma(\text{notas}[i] \geq 7.0)$
Conteo de reprobados	Frecuencia absoluta con nota < 7	$\Sigma(\text{notas}[i] < 7.0)$

Estas métricas proporcionan una visión global del rendimiento académico del grupo y constituyen la base para decisiones pedagógicas informadas por parte del docente.

3.6 Pseudocódigo y Diagramas de Flujo

El pseudocódigo es una representación semi-formal de un algoritmo que combina lenguaje natural con convenciones de programación, actuando como puente entre el análisis conceptual y la implementación concreta [1]. No está ligado a ningún lenguaje de programación específico, lo que facilita su comprensión universal.

El diagrama de flujo es la representación gráfica equivalente, empleando símbolos estandarizados por la norma ISO 5807: óvalo (inicio/fin), rectángulo (proceso), rombo (decisión), paralelogramo (entrada/salida) y flechas (flujo de control) [6]. Ambas herramientas son componentes esenciales de la etapa de diseño en cualquier metodología de desarrollo de software.

4. ANÁLISIS DEL PROBLEMA

4.1 Descripción del Problema

Una institución educativa necesita automatizar el procesamiento de las calificaciones del examen final de sus estudiantes. Actualmente, este proceso se realiza de forma manual, generando demoras, errores aritméticos y dificultad para obtener información estadística consolidada o consultar el estado de un



estudiante específico. Se requiere un programa de consola en C++ que resuelva estas necesidades de manera eficiente y confiable.

4.2 Entradas del Sistema

#	Dato	Tipo	Descripción
1	n	int	Número total de estudiantes a registrar
2	nombres[i]	string	Nombre completo de cada estudiante
3	notas[i]	float	Calificación del examen final (rango: 0.0 – 10.0)
4	nombreBuscar	string	Nombre del estudiante a localizar en la búsqueda
5	opcion	int	Selección del menú principal (0 – 3)

4.3 Procesos del Sistema

#	Proceso	Módulo
1	Almacenar nombre y nota de cada estudiante en arreglos paralelos	registrarCalificaciones()
2	Sumar todas las notas y dividir para n	mostrarReporte()
3	Comparar cada nota con el umbral 7.0 y contabilizar	mostrarReporte()
4	Recorrer nombres[] comparando con el criterio de búsqueda	buscarEstudiante()
5	Determinar el estado de aprobación del estudiante hallado	buscarEstudiante()

4.4 Salidas del Sistema

#	Salida	Condición
1	Confirmación de registro completado	Siempre tras registrar
2	Promedio general del grupo	Reporte estadístico
3	Número de estudiantes aprobados y reprobados	Reporte estadístico
4	Nombre, nota y estado del estudiante	Si la búsqueda tiene éxito
5	"El estudiante [X] no se encuentra registrado"	Si la búsqueda falla

4.5 Restricciones del Sistema

- El tamaño máximo del arreglo se fija en MAX = 50 estudiantes.
- Las calificaciones válidas están en el rango [0.0, 10.0].
- La búsqueda es *case-sensitive* (distingue mayúsculas de minúsculas).



- No se puede mostrar el reporte ni realizar búsquedas sin haber registrado datos previamente.
- El menú debe rechazar opciones fuera del rango [0, 3].

5. TABLA DE VARIABLES EN C++

Variable	Tipo C++	Descripción	Alcance	Valor Inicial
MAX	const int	Tamaño máximo del arreglo de estudiantes	Global	50
nombres[]	string[MAX]	Arreglo de nombres de estudiantes	Global	"" (vacío)
notas[]	float[MAX]	Arreglo de calificaciones paralelo a nombres[]	Global	0.0
n	int	Número real de estudiantes registrados	Global / main()	0
i	int	Índice de iteración en bucles for	Local (funciones)	0
suma	float	Acumulador de notas para el cálculo del promedio	Local mostrarReporte()	0.0
promedio	float	Promedio general del grupo (suma / n)	Local mostrarReporte()	0.0
aprobados	int	Contador de estudiantes con notas[i] >= 7.0	Local mostrarReporte()	0
reprobados	int	Contador de estudiantes con notas[i] < 7.0	Local mostrarReporte()	0
nombreBuscar	string	Nombre ingresado para la búsqueda secuencial	Local buscarEstudiante()	""
encontrado	bool	Bandera: true si el estudiante fue hallado	Local buscarEstudiante()	false
estado	string	Cadena "Aprobado" o "Reprobado" según la nota	Local buscarEstudiante()	""
opcion	int	Opción seleccionada en el menú principal	Local main()	0

Nota: Los arreglos nombres[] y notas[] se declaran globalmente con tamaño MAX. Los índices comienzan en 0 conforme a la convención estándar de C++. La variable n controla la cantidad efectiva de registros válidos.

6. ALGORITMO LÓGICO

El siguiente algoritmo describe de forma secuencial y estructurada el camino completo para resolver el problema, desglosado por módulos funcionales.



6.1 Algoritmo General — Menú Principal

INICIO

DECLARAR nombres[50] : cadena

DECLARAR notas[50] : real

DECLARAR n : entero $\leftarrow 0$

DECLARAR opcion : entero

REPETIR

MOSTRAR "===== MENÚ PRINCIPAL ====="

MOSTRAR "1. Registrar calificaciones"

MOSTRAR "2. Mostrar reporte estadístico"

MOSTRAR "3. Buscar estudiante"

MOSTRAR "0. Salir"

LEER opcion

SEGÚN opcion HACER

CASO 1: LLAMAR registrarCalificaciones()

CASO 2: LLAMAR mostrarReporte()

CASO 3: LLAMAR buscarEstudiante()

CASO 0: MOSTRAR "Saliendo del sistema..."

OTRO: MOSTRAR "Opción no válida. Intente nuevamente."

FIN SEGÚN

HASTA QUE opcion = 0

FIN

6.2 Módulo 1 — Registrar Calificaciones

FUNCIÓN registrarCalificaciones()

PASO 1: Solicitar el número de estudiantes

MOSTRAR "Ingrese el número de estudiantes:"

LEER n

VALIDAR que $n > 0$ y $n \leq 50$

PASO 2: Iniciar ciclo de registro

PARA i DESDE 0 HASTA n-1 HACER

PASO 3: Ingresar nombre del estudiante i

MOSTRAR "Ingrese nombre del estudiante", (i+1), ":"

LEER nombres[i]

PASO 4: Ingresar calificación del estudiante i

MOSTRAR "Ingrese nota del estudiante", (i+1), ":"

LEER notas[i]

VALIDAR que $\text{notas}[i] \geq 0.0$ y $\text{notas}[i] \leq 10.0$



FIN PARA

PASO 5: Confirmar registro exitoso

MOSTRAR "Registro completado. Total de estudiantes:", n

FIN FUNCIÓN

6.3 Módulo 2 — Mostrar Reporte Estadístico

FUNCIÓN mostrarReporte()

PASO 1: Verificar que existan datos registrados

SI $n = 0$ ENTONCES

MOSTRAR "Error: No hay estudiantes registrados."

RETORNAR

FIN SI

PASO 2: Inicializar variables acumuladoras

suma $\leftarrow 0.0$

aprobados $\leftarrow 0$

reprobados $\leftarrow 0$

PASO 3: Recorrer el arreglo de notas

PARA i DESDE 0 HASTA $n-1$ HACER

PASO 4: Acumular la nota en suma

suma $\leftarrow$ suma + notas[i]

PASO 5: Clasificar al estudiante

SI notas[i] ≥ 7.0 ENTONCES

aprobados $\leftarrow$ aprobados + 1

SINO

reprobados $\leftarrow$ reprobados + 1

FIN SI

FIN PARA

PASO 6: Calcular el promedio general

promedio $\leftarrow$ suma / n

PASO 7: Presentar el reporte en pantalla

MOSTRAR "===== REPORTE ESTADÍSTICO ====="

MOSTRAR "Promedio general :", promedio

MOSTRAR "Total aprobados :", aprobados

MOSTRAR "Total reprobados :", reprobados

FIN FUNCIÓN

6.4 Módulo 3 — Búsqueda Secuencial de Estudiante



FUNCIÓN buscarEstudiante()

PASO 1: Verificar que existan datos registrados

SI $n = 0$ ENTONCES

MOSTRAR "Error: No hay estudiantes registrados."

RETORNAR

FIN SI

PASO 2: Solicitar el nombre a buscar

MOSTRAR "Ingrese el nombre del estudiante a buscar:"

LEER nombreBuscar

PASO 3: Inicializar bandera de búsqueda

encontrado $\leftarrow$ FALSO

PASO 4: Recorrer secuencialmente el arreglo de nombres

PARA i DESDE 0 HASTA $n-1$ HACER

PASO 5: Comparar el nombre actual con el criterio de búsqueda

SI nombres[i] = nombreBuscar ENTONCES

PASO 6: Determinar el estado del estudiante encontrado

SI notas[i] ≥ 7.0 ENTONCES

estado $\leftarrow$ "Aprobado"

SINO

estado $\leftarrow$ "Reprobado"

FIN SI

PASO 7: Mostrar los datos del estudiante

MOSTRAR "Nombre :", nombres[i]

MOSTRAR "Nota :", notas[i]

MOSTRAR "Estado :", estado

PASO 8: Marcar como encontrado y detener la búsqueda

encontrado $\leftarrow$ VERDADERO

SALIR del bucle (break)

FIN SI

FIN PARA

PASO 9: Evaluar resultado final de la búsqueda

SI encontrado = FALSO ENTONCES

MOSTRAR "El estudiante", nombreBuscar, "no se encuentra registrado."

FIN SI

FIN FUNCIÓN



6.5 Código fuente en Pseint

```
Proceso Gestion_Calificaciones
// Definición de variables
Definir n, i, aprobados, reprobados Como Entero
Definir suma, promedio, notas Como Real
Definir nombres Como Cadena
Definir buscar Como Cadena
Definir encontrado Como Logico

// Dimensionamiento de arreglos (vectores)
Dimension nombres[100]
Dimension notas[100]

// Inicialización de variables
suma <- 0
aprobados <- 0
reprobados <- 0

// Entrada de datos del total de estudiantes
Escribir "Ingrese la cantidad de estudiantes:"
Leer n

// Ciclo para registrar los datos de cada estudiante
Para i <- 1 Hasta n Hacer
    Escribir "Ingrese el nombre del estudiante ", i, ":"
    Leer nombres[i]

    Escribir "Ingrese la nota del estudiante ", i, ":"
    Leer notas[i]

    suma <- suma + notas[i]

// Validación de aprobación (nota mínima: 7)
Si notas[i] >= 7 Entonces
    aprobados <- aprobados + 1
Sino
    reprobados <- reprobados + 1
FinSi
FinPara

// Cálculo del promedio
promedio <- suma / n

// Despliegue de resultados estadísticos
Escribir " "
Escribir "===== REPORTE ESTADISTICO ====="
```



```
Escribir "Promedio general: ", promedio
Escribir "Aprobados: ", aprobados
Escribir "Reprobados: ", reprobados
Escribir "===== "
Escribir " "
```

```
// Módulo de búsqueda de estudiantes
Escribir "Ingrese el nombre del estudiante a buscar:"
Leer buscar
```

```
encontrado <- Falso
```

```
// Ciclo de búsqueda en el arreglo
Para i <- 1 Hasta n Hacer
    Si nombres[i] = buscar Entonces
        encontrado <- Verdadero
        Escribir " "
        Escribir "--- Estudiante Encontrado ---"
        Escribir "Nombre: ", nombres[i]
        Escribir "Nota: ", notas[i]
```

```
    Si notas[i] >= 7 Entonces
        Escribir "Estado: Aprobado"
    Sino
        Escribir "Estado: Reprobado"
    FinSi
    Escribir "-----"
```

```
FinSi
FinPara
```

```
// Mensaje en caso de no encontrar al estudiante
Si encontrado = Falso Entonces
    Escribir "El estudiante no se encuentra registrado."
```

```
FinSi
```

```
FinProceso
```

6.6 Código Fuente en C++

```
#include <iostream>
#include <string>
using namespace std;

// — Constante y variables globales
```



```
const int MAX = 50;
string nombres[MAX];
float notas[MAX];
int n = 0;
```

```
// — Módulo 1: Registrar calificaciones
```

```
void registrarCalificaciones() {
    cout << "\nIngrese el numero de estudiantes: ";
    cin >> n;
    cin.ignore();

    for (int i = 0; i < n; i++) {
        cout << "\nEstudiante " << (i + 1) << ": \n";
        cout << " Nombre : ";
        getline(cin, nombres[i]);
        cout << " Nota : ";
        cin >> notas[i];
        cin.ignore();
    }
    cout << "\n>> Registro completado. Total de estudiantes: " << n << "\n";
}
```

```
// — Módulo 2: Reporte estadístico
```

```
void mostrarReporte() {
    if (n == 0) { cout << ">> Error: No hay estudiantes registrados.\n"; return; }

    float suma = 0.0;
    int aprobados = 0, reprobados = 0;

    for (int i = 0; i < n; i++) {
        suma += notas[i];
        if (notas[i] >= 7.0) aprobados++;
        else reprobados++;
    }

    cout << "\n===== REPORTE ESTADÍSTICO =====\n";
    cout << " Promedio general : " << suma / n << "\n";
    cout << " Aprobados : " << aprobados << "\n";
    cout << " Reprobados : " << reprobados << "\n";
}
```

```
// — Módulo 3: Búsqueda secuencial
```

```
void buscarEstudiante() {
    if (n == 0) { cout << ">> Error: No hay estudiantes registrados.\n"; return; }

    string buscar;
    bool encontrado = false;

    cout << "\nIngrese el nombre del estudiante a buscar: ";
```



```
cin.ignore();
getline(cin, buscar);

for (int i = 0; i < n; i++) {
    if (nombres[i] == buscar) {
        string estado = (notas[i] >= 7.0) ? "Aprobado" : "Reprobado";
        cout << "\n-- Estudiante encontrado --\n";
        cout << " Nombre : " << nombres[i] << "\n";
        cout << " Nota   : " << notas[i]   << "\n";
        cout << " Estado : " << estado      << "\n";
        encontrado = true;
        break;
    }
}

if (!encontrado)
    cout << "\n>> El estudiante \"" << buscar << "\" no se encuentra registrado.\n";
}

// — Programa principal

int main() {
    int opcion;
    do {
        cout << "\n===== MENU PRINCIPAL =====\n"
            << " 1. Registrar calificaciones\n"
            << " 2. Mostrar reporte estadístico\n"
            << " 3. Buscar estudiante\n"
            << " 0. Salir\n"
            << "Seleccione una opcion: ";
        cin >> opcion;

        switch (opcion) {
            case 1: registrarCalificaciones(); break;
            case 2: mostrarReporte();          break;
            case 3: buscarEstudiante();         break;
            case 0: cout << "Saliendo...\n";    break;
            default: cout << ">> Opcion no valida.\n";
        }
    } while (opcion != 0);

    return 0;
}
```

7. RESULTADOS OBTENIDOS

7.1 Datos de Prueba

Para verificar el correcto funcionamiento del sistema se utilizó el siguiente conjunto de datos de prueba con **5 estudiantes**:



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



#	Nombre	Nota	Estado esperado
1	Ana García	9.5	Aprobado
2	Luis Mora	5.0	Reprobado
3	Sofía Pérez	8.0	Aprobado
4	Juan Tito	4.5	Reprobado
5	María León	7.0	Aprobado

7.2 Prueba de Escritorio — Módulo de Reporte

Traza de ejecución del bucle de cálculo estadístico:

Iteración `i`	`notas[i]`	`suma` (acum.)	`aprobados`	`reprobados`
0	9.5	9.5	1	0
1	5.0	14.5	1	1
2	8.0	22.5	2	1
3	4.5	27.0	2	2
4	7.0	34.0	3	2

Cálculo del promedio: $34.0 / 5 = 6.8$

Reporte generado por el sistema:

```
===== REPORTE ESTADÍSTICO =====
Promedio general : 6.8
Aprobados       : 3
Reprobados      : 2
```

7.3 Prueba de Escritorio — Módulo de Búsqueda

Caso 1 — Estudiante encontrado (búsqueda: "Luis Mora"):

Paso	`i`	`nombres[i]`	Comparación	`encontrado`
Inicialización	—	—	—	false
Iteración i=0	0	"Ana García"	$\neq$ "Luis Mora"	false
Iteración i=1	1	"Luis Mora"	= "Luis Mora"	true → break

Salida obtenida:

```
-- Estudiante encontrado --
Nombre : Luis Mora
Nota   : 5.0
Estado : Reprobado
```



Caso 2 — Estudiante no encontrado (búsqueda: "Carlos Ruiz"):

El bucle recorre los 5 registros sin encontrar coincidencia. encontrado permanece false.

Salida obtenida:

```
>> El estudiante "Carlos Ruiz" no se encuentra registrado.
```

7.4 Validación de Resultados

Indicador	Valor esperado	Valor obtenido	¿Correcto?
Promedio general	6.8	6.8	Si
Total aprobados	3	3	Si
Total reprobados	2	2	Si
Búsqueda exitosa (Luis Mora)	Nota: 5.0 / Reprobado	Nota: 5.0 / Reprobado	Si
Búsqueda fallida (Carlos Ruiz)	Mensaje de error	Mensaje de error	Si

Todos los indicadores de prueba fueron superados satisfactoriamente, confirmando la corrección lógica del algoritmo implementado.

8. CONCLUSIONES

- El análisis previo del problema resultó determinante para identificar con exactitud las tres entidades del sistema (entradas, procesos y salidas). La definición anticipada de las variables y sus tipos en C++ evitó ambigüedades en la codificación y garantizó la coherencia entre el diseño y la implementación final.
- El diseño del algoritmo mediante pasos secuenciales y el pseudocódigo por módulos demostró ser una etapa indispensable del ciclo de desarrollo. Esta fase permitió detectar condiciones de borde —como el intento de reportar sin datos registrados— antes de escribir una sola línea de código, reduciendo significativamente el tiempo de depuración.
- La implementación en C++ mediante arreglos paralelos y funciones modularizadas resultó exitosa y produjo código limpio, legible y mantenible. La separación de responsabilidades entre registrarCalificaciones(), mostrarReporte() y buscarEstudiante() facilita futuras extensiones del sistema sin afectar los módulos existentes.
- El algoritmo de búsqueda secuencial cumple adecuadamente su función para el volumen de datos esperado en un entorno académico. Si bien su complejidad $O(n)$ no es óptima para conjuntos masivos de datos, su simplicidad de implementación y la ausencia de requisito de ordenamiento previo lo hacen la elección correcta para este contexto.
- Las pruebas de escritorio confirmaron la corrección total del programa. Los cinco indicadores de validación —promedio, conteo de aprobados, conteo de reprobados, búsqueda exitosa y búsqueda fallida— arrojaron los resultados esperados, validando la integridad lógica del sistema desarrollado.



9. RECOMENDACIONES

- **Validación de entradas:** Se recomienda implementar rutinas de validación que rechacen calificaciones fuera del rango [0.0, 10.0] y valores de n negativos o superiores a MAX. Esto incrementa la robustez del sistema frente a errores del usuario sin modificar la lógica central.
- **Búsqueda insensible a mayúsculas:** La búsqueda actual es *case-sensitive*, lo que puede generar falsos negativos por errores tipográficos. Se recomienda normalizar ambas cadenas a minúsculas —usando `tolower()` o `transform()` de la STL— antes de la comparación, mejorando notablemente la usabilidad.
- **Migración a estructuras dinámicas:** Para eliminar la restricción del tamaño fijo de 50 estudiantes, se recomienda migrar de arreglos estáticos a `vector<string>` y `vector<float>` de la STL, lo que permite gestionar colecciones de tamaño variable en tiempo de ejecución sin cambios en la lógica del programa.
- **Persistencia de datos:** En su estado actual el sistema pierde todos los datos al cerrarse. Se recomienda incorporar persistencia mediante archivos de texto (usando `fstream`) o una base de datos embebida como SQLite, lo que transformaría el sistema en una herramienta funcional para uso institucional real.
- **Aplicación de la metodología completa:** Se recomienda mantener la práctica de desarrollar análisis, pseudocódigo, diagrama de flujo y pruebas de escritorio en todos los proyectos de programación futuros. Esta disciplina metodológica, aunque parezca costosa en tiempo inicial, reduce errores de lógica, facilita el trabajo en equipo y constituye la base de metodologías de ingeniería de software más avanzadas como el ciclo de vida del software (SDLC) y las metodologías ágiles.

10. REFERENCIAS BIBLIOGRÁFICAS

- [1] L. Joyanes Aguilar, *Fundamentos de Programación: Algoritmos, Estructura de Datos y Objetos*, 4.^a ed. Madrid, España: McGraw-Hill, 2008.
- [2] B. Stroustrup, *The C++ Programming Language*, 4th ed. Upper Saddle River, NJ, USA: Addison-Wesley Professional, 2013.
- [3] P. Deitel y H. Deitel, *C++ How to Program*, 10th ed. Hoboken, NJ, USA: Pearson Education, 2017.
- [4] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.
- [5] M. R. Spiegel y L. J. Stephens, *Estadística*, 4.^a ed. México D.F., México: McGraw-Hill, 2009.
- [6] International Organization for Standardization, *ISO 5807:1985 — Information Processing: Documentation Symbols and Conventions for Data, Program and System Flowcharts, Program Network Charts and System Resources Charts*, Ginebra, Suiza: ISO, 1985.
- [7] S. Prata, *C++ Primer Plus*, 6th ed. Upper Saddle River, NJ, USA: Addison-Wesley Professional, 2012.
- [8] R. Sedgewick y K. Wayne, *Algorithms*, 4th ed. Upper Saddle River, NJ, USA: Addison-Wesley Professional, 2011.
- [9] isma1402uta-boop, "T2.7.-Arreglos-Ejercicios: T2.7. arreglos ejercicios," GitHub repository, 2026. [Online]. Available: <https://github.com/isma1402uta-boop/T2.7.-Arreglos-Ejercicios>. [Accessed: 27-May-2026].



11. ANEXOS

11.1 Diagrama de flujo del ejercicio planteado

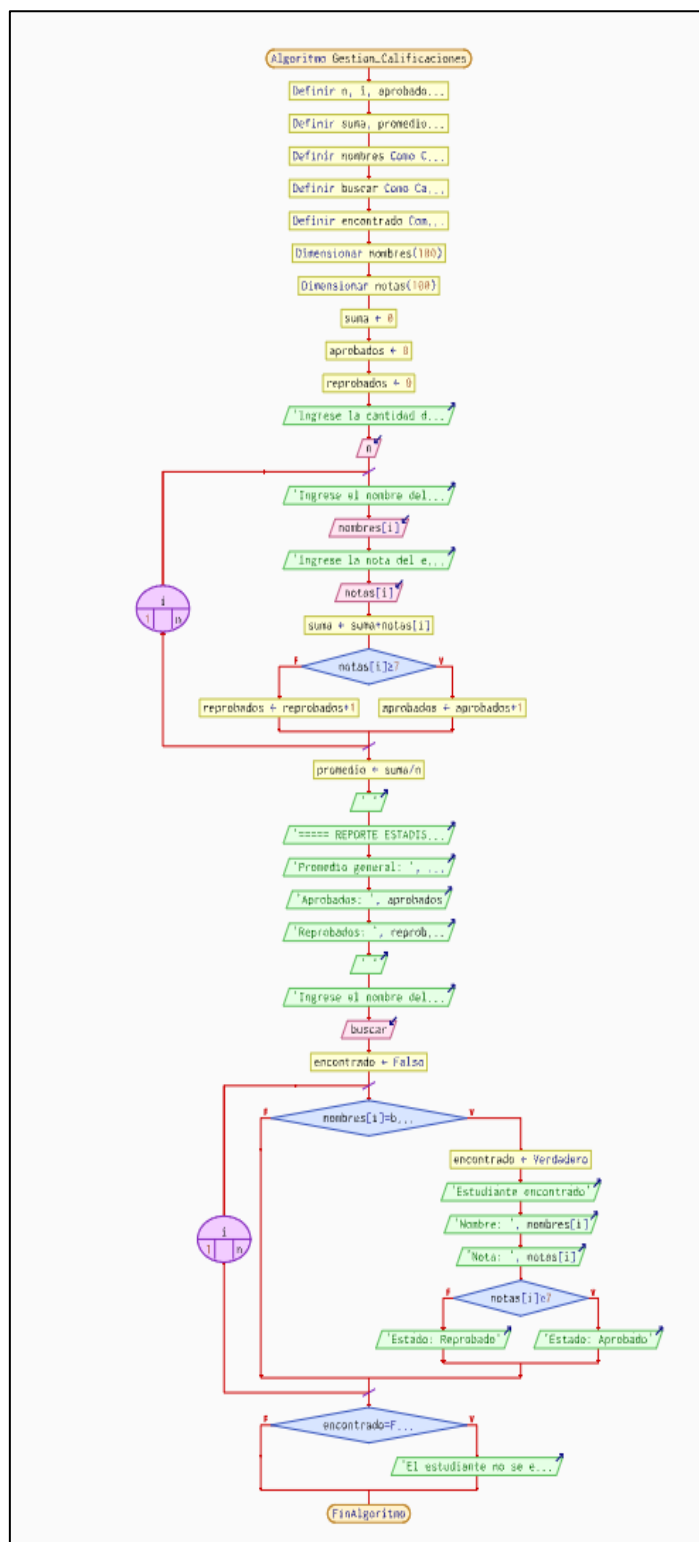


Fig. 1. Representación gráfica del proceso para la gestión y búsqueda de calificaciones.
Elaboración propia.



11.2 Menú del ejercicio planteado

```
===== MENU PRINCIPAL =====  
1. Registrar calificaciones  
2. Mostrar reporte estadístico  
3. Buscar estudiante  
0. Salir  
Seleccione una opcion:
```

Fig. 2. Menú principal dentro del proceso para la gestión y búsqueda de calificaciones.
Elaboración propia.

11.3 Registro de calificaciones dentro del sistema

```
===== MENU PRINCIPAL =====  
1. Registrar calificaciones  
2. Mostrar reporte estadístico  
3. Buscar estudiante  
0. Salir  
Seleccione una opcion: 1  
  
Ingrese el numero de estudiantes: 2  
  
Estudiante 1:  
Nombre : Sanchez Isaac  
Nota   : 10  
  
Estudiante 2:  
Nombre : Garcia Melany  
Nota   : 10  
  
>> Registro completado. Total de estudiantes: 2
```

Fig. 3. Registro dentro del proceso para la gestión y búsqueda de calificaciones.
Elaboración propia.

11.4 Reporte dentro del sistema de registro de estudiantes

```
===== REPORTE ESTADÍSTICO =====  
Promedio general : 10  
Aprobados        : 2  
Reprobados       : 0
```

Fig. 4. Reporte estadístico dentro del proceso para la gestión y búsqueda de calificaciones.
Elaboración propia.

Fig. 6. Infografía que represente el funcionamiento del código desarrollado.
Elaboración propia